

Creational Pattern Assignment

AI Chat Link: <https://chatgpt.com/share/6a402982-1c44-83ee-ba0c-06c225c033e2>

Summary of AI Assistance

During the development of the Exam Management System assignment, I used ChatGPT to understand the required design patterns, Java concepts, and UML representation. The discussions covered the following topics:

1. Java Classes and Packages

- Whether every class requires a separate .java file.
- How Java packages work.
- When to organize classes into different packages.
- Suggested package structure for the assignment.

2. Factory Method Pattern

- Understanding the purpose of the Factory Method Pattern.
- Relationship between the Exam interface and the ExamFactory abstract class.
- Implementing PracticeQuiz, MidtermExam, FinalExam, and their respective factories.
- Why the factory returns an Exam reference instead of a concrete class.

3. Interfaces and Abstract Classes

- Difference between implements and extends.
- Why interface methods are automatically public abstract.
- Why implemented methods must also be declared public.
- Difference between interfaces and abstract classes.

4. Abstract Factory Pattern

- Creating families of related objects:
 - Questions
 - Renderers
 - Evaluators
- Why renderers and evaluators receive the interface type (Question) instead of concrete classes.
- How Abstract Factory differs from Factory Method.

5. Builder Pattern

- Understanding the Builder Pattern.
- Method chaining using return this.
- Why return this does not create new objects.
- Difference between storing individual fields in the builder and storing a single ExamConfiguration object.
- Implementing the build() method.

6. Exam Configuration

- Purpose of the ExamConfiguration class.
- Relationship between Exam and ExamConfiguration.
- How configuration details can be displayed.
- Whether displayInfo() belongs in Exam or ExamConfiguration.

7. Dependency Injection and Inversion of Control

- Meaning of Dependency Injection (DI).
- Meaning of Inversion of Control (IoC).
- Constructor injection using the QuestionSource interface.
- Why factories should receive dependencies instead of creating them internally.
- Advantages of loose coupling.

8. Question Sourcing

- Purpose of the QuestionSource interface.
- Difference between BankQuestionSource and FakerQuestionSource.
- How QuestionSource integrates with the Abstract Factory.
- Where question data should originate.

9. Question Storage

- Where to store question text.
- Where to store question points and difficulty.
- Suggested implementation of a QuestionBank class containing predefined questions.
- Why factories should not hard-code questions.

10. System Execution Workflow

- Understanding the required sequence in Main.java:
 1. Create the appropriate ExamFactory.
 2. Select a QuestionSource.
 3. Inject the source into question factories.
 4. Build the exam configuration using ExamBuilder.
 5. Print the final dashboard.

11. Console Output Design

- How to generate the required dashboard.
- Printing exam information.
- Printing configuration settings.
- Printing question source information.
- Printing question type, points, and difficulty.

12. UML Class Diagram

- UML notation for interfaces.
- UML notation for abstract classes.
- Difference between implements and extends relationships.
- Whether inherited methods need to be repeated in subclasses (generally not required unless specifically requested).

13. Assignment Design Discussion

- Discussed how the four design patterns (Factory Method, Abstract Factory, Builder, and Dependency Injection) fit together.
- Identified places where the assignment specification leaves implementation details open (for example, how QuestionSource selects question types and how question metadata is represented).
- Planned a practical implementation that follows the assignment requirements while keeping the design organized.

This summarizes the concepts and implementation decisions discussed while completing the assignment.